Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:02/04/2026**	**Enrolment No:92301733054**

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

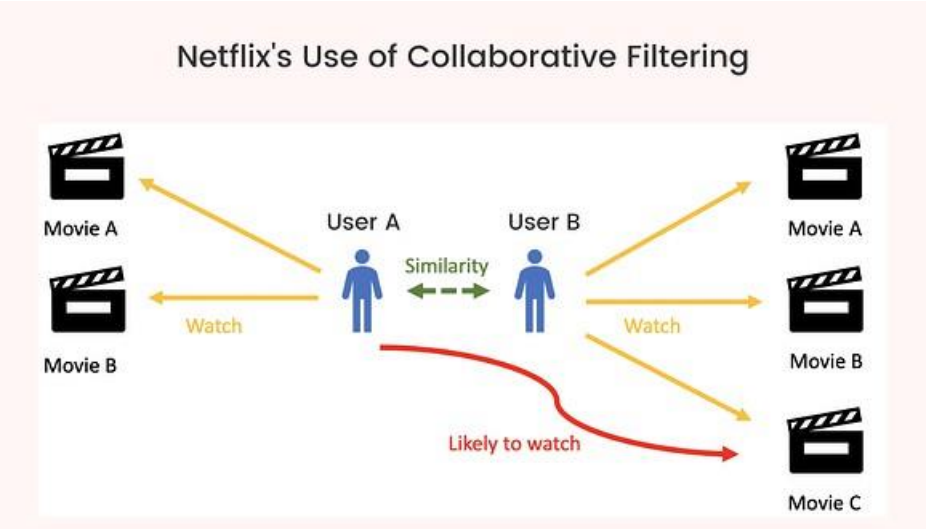
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



Pre Lab Exercise:

1. **When can you use collaborative-based recommendation system approach?**
   - When user interaction data (ratings, clicks, history) is available
   - When you want to recommend based on similar users' preferences
   - When item features are not available or not required

- When there are many users and sufficient data

2. **Write advantages of collaborative -based filtering approach**

- Better personalization
- Recommends diverse and new items
- No need for item features
- Captures user behavior effectively

3. **Write disadvantages of collaborative -based filtering approach**

- Cold-start problem
- Requires large data
- Data sparsity issue
- Computationally expensive

Program (Code):

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("ratings.csv")
# ================= USER-ITEM MATRIX =================
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)
# ================= USER SIMILARITY =================
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ================= RECOMMEND FUNCTION =================
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])

    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values
```

```
    # Normalize
    weighted_ratings /= similar_users.sum()

    # Get unseen movies
    user_rated = user_item_matrix.loc[user_id]
    unseen_movies = user_rated[user_rated == 0].index

    scores = list(zip(unseen_movies,
weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    recommended_ids = [movie for movie, _ in scores[:top_n]]

    return recommended_ids
print(recommend_movies(user_id=1))
```

Results:

```
... [1200, 1610, 541, 589, 1036]
```

Observation: We learned this experiment to recommend movies using a collaborative filtering approach based on user preferences. The system finds similar users using cosine similarity and suggests movies liked by them. The results show that recommendations depend on user behavior and are more diverse compared to content-based filtering.

Post Lab Exercise:

1. **Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.**

   Yes, the results are satisfactory because the system recommends movies based on similar users' preferences, which improves personalization. It provides diverse recommendations compared to content-based methods. However, performance may be affected when there is insufficient user data or sparse ratings, which can reduce accuracy.

2. **Recommend top-10 "movies" using MovieLens 100K dataset (Link: https://grouplens.org/datasets/movielens/100k/). Attach the code with output. Also, write your observation for the same.**

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:02/04/2026**	**Enrolment No:92301733054**

Program Code:

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

ratings = pd.read_csv("ratings.csv")
movies = pd.read_csv("movies.csv")

user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)

user_similarity = cosine_similarity(user_item_matrix)

user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)

def recommend_movies(user_id, top_n=10):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    weighted_ratings = np.zeros(user_item_matrix.shape[1])

    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values

    weighted_ratings /= similar_users.sum()

    user_rated = user_item_matrix.loc[user_id]
    unseen_movies = user_rated[user_rated == 0].index
```

```
    scores = list(zip(
        unseen_movies,
        weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]
    ))

    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    recommended_ids = [movie for movie, _ in scores[:top_n]]

    return movies[movies["movieId"].isin(recommended_ids)][["title", "genres"]]

print(recommend_movies(user_id=1, top_n=10))
```

Result:

```
recommend_movies(user_id=1, top_n=10)
```

	title	genres
292	Pulp Fiction (1994)	Comedy|Crime|Drama|Thriller
840	Godfather, The (1972)	Crime|Drama
882	Vertigo (1958)	Drama|Mystery|Romance|Thriller
898	Wizard of Oz, The (1939)	Adventure|Children|Fantasy|Musical
899	Gone with the Wind (1939)	Drama|Romance|War
903	2001: A Space Odyssey (1968)	Adventure|Drama|Sci-Fi
932	It's a Wonderful Life (1946)	Children|Drama|Fantasy|Romance
1164	One Flew Over the Cuckoo's Nest (1975)	Drama
1323	Sling Blade (1996)	Drama
2766	American Beauty (1999)	Drama|Romance